

Computer Vision Foundations

Convolutional Neural Network

Created By : Abdan Hafidz

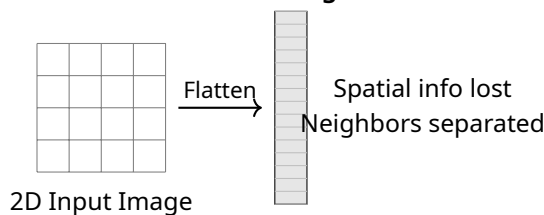
1 Introduction: The Spatial Inductive Bias

When we open our eyes, we do not perceive the world as a flattened stream of data points. We see edges, textures, shapes, and spatial relationships. A cat in the top-left corner of our vision is structurally identical to a cat in the bottom-right corner. This concept is known as *translation invariance*, and it is the fundamental motivation behind the Convolutional Neural Network (CNN).

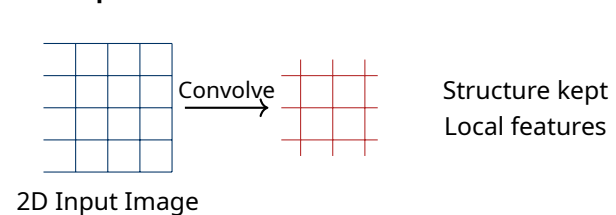
Traditional Multi-Layer Perceptrons (MLPs) struggle profoundly with image data. By flattening an image into a 1D vector, the network permanently destroys the 2D local structure. The pixel at coordinate $(0, 0)$ and $(0, 1)$ become just two numbers in a long list, with no inherent relationship preserved.

To solve this, CNNs introduce a strong *inductive bias*: we assume that features are local and hierarchical.

MLP: Destructive Flattening



CNN: Spatial Preservation



2 The Mathematics of Convolution

Despite the name, what deep learning frameworks typically implement is technically known as *cross-correlation*. In the discrete domain of digital images, we operate on a grid. Let us formalize the image as a matrix I of dimensions $H \times W$ and a learnable filter (or kernel) K of dimensions $k_h \times k_w$.

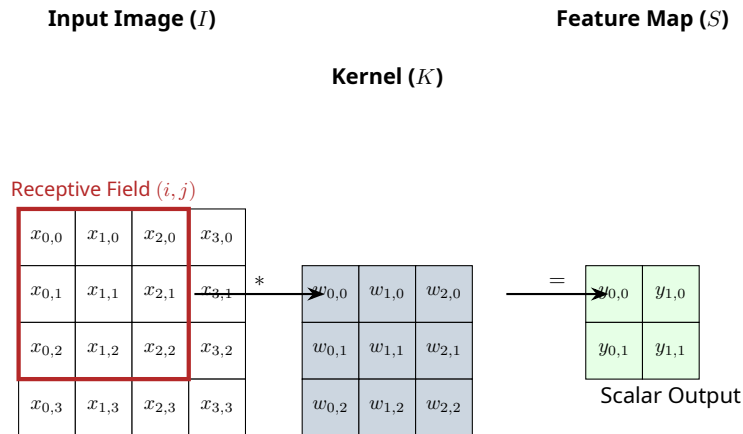
The rigorous mathematical definition for the value of the output feature map S at position (i, j) is derived through an accumulation of overlapping signals.

Deriving the Formula Step-by-Step:

1. **Positioning:** We align the top-left corner of the kernel K with the pixel (i, j) of the image I .
2. **Scanning:** The kernel has a width and height. We must iterate over every cell inside the kernel to calculate the match. We use indices m and n to represent the offset within the kernel itself.
3. **Matching:** At each kernel offset (m, n) , we look at the image pixel $(i+m, j+n)$. We multiply the kernel weight $K(m, n)$ by the image pixel $I(i+m, j+n)$.
4. **Accumulation:** We sum all these products together to get a single value.

This logic creates the standard 2D convolution formula:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i+m, j+n) K(m, n) \quad (1)$$



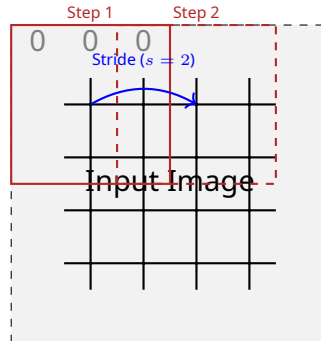
2.1 Weight Sharing

The profound efficiency of CNNs arises from parameter sharing. If the kernel K is designed to detect a vertical edge, we use the exact same K (the same 9 numbers in a 3×3 grid) to look at the top-left and the bottom-right of the image. We do not learn new weights for different locations.

3 Hyperparameters: Stride and Padding

Two geometric constraints govern the dimensions of the output feature map: Stride and Padding.

Padding (p): Without padding, a kernel cannot center itself on the edge pixels. We often “pad” the input with zeros around the border to maintain dimensions. **Stride (s):** This controls the step size. A stride of 1 moves one pixel; a stride of 2 skips pixels, downsampling the image.



Step-by-Step Derivation of Output Dimensions: How do we calculate the width of the next layer (W_{out})?

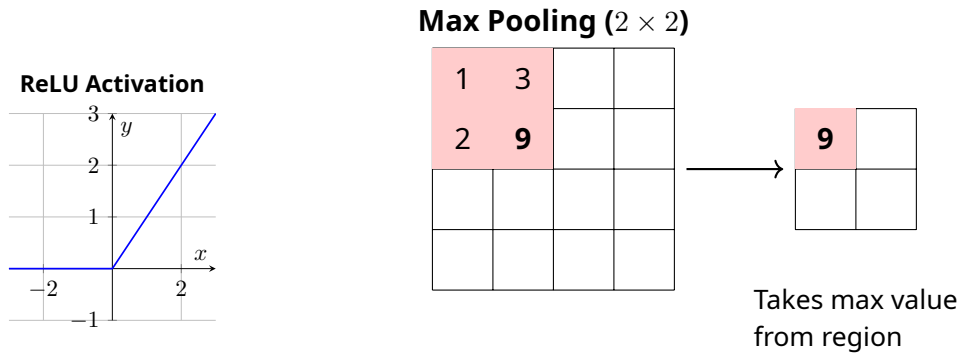
1. Start with the input width: W_{in} .
2. Add the extra pixels created by padding on both sides: $W_{in} + 2p$.
3. Subtract the size of the kernel (k), because the kernel's right edge hits the image boundary before its center does: $(W_{in} + 2p) - k$.
4. Divide by the stride (s) to account for skipping pixels: $\frac{W_{in} + 2p - k}{s}$.
5. Add 1, because the number of steps is the intervals plus the starting position (the “fence-post” rule):

$$W_{out} = \left\lfloor \frac{W_{in} - k + 2p}{s} \right\rfloor + 1 \quad (2)$$

4 Non-Linearity and Pooling

A convolution is a linear transformation. To approximate complex functions, we introduce non-linearity via activation functions like ReLU (Rectified Linear Unit), and we reduce dimensionality via Pooling.

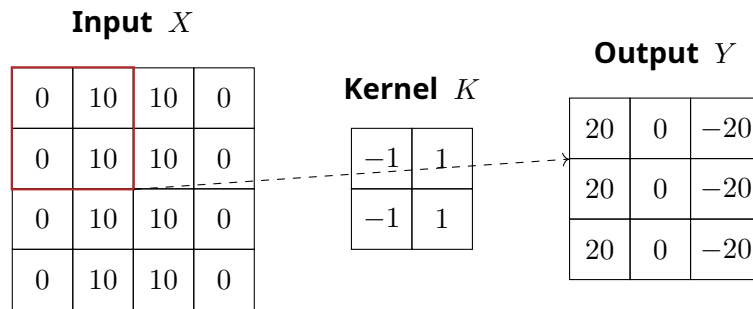
ReLU ($f(x) = \max(0, x)$): It replaces all negative pixel values with zero. This adds non-linearity without affecting positive features. **Max Pooling:** Summarizes a region by picking its strongest feature.



5 Case Study: Edge Detection Calculation

To ground these concepts, let us perform a manual forward pass for vertical edge detection.

Scenario: A 4×4 image with a vertical boundary (left is dark/0, right is light/10). We apply a 2×2 vertical edge filter.



5.1 Calculation Walkthrough

1. **Patch 1 (Left):** We take the first 2×2 block: $\begin{bmatrix} 0 & 10 \\ 0 & 10 \end{bmatrix}$. We multiply element-wise with K :
 $(0 \cdot -1) + (10 \cdot 1) + (0 \cdot -1) + (10 \cdot 1) = 20$.
2. **Patch 2 (Middle):** Shift right by 1 pixel: $\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$. Result: $(10 \cdot -1) + (10 \cdot 1) + (10 \cdot -1) + (10 \cdot 1) = 0$. (No change seen).
3. **Patch 3 (Right):** Shift right again: $\begin{bmatrix} 10 & 0 \\ 10 & 0 \end{bmatrix}$. Result: $(10 \cdot -1) + (0 \cdot 1) + (10 \cdot -1) + (0 \cdot 1) = -20$. (Inverse edge).

6 Modern Context

Modern architectures like ResNet and EfficientNet scale these principles to massive depths. Yet, regardless of complexity, the fundamental logic remains: a learnable hierarchy of local features, progressively aggregating simple edges into complex semantic concepts.